

Laboratorio 5

Información

Integrantes:

Name	Institution ID	GitHub User
Josué Say	22801	JosueSay
Carlos Valladares	221164	vgcarlol

- [Repositorio](#)

Preparación de entorno

```
In [4]: # %pip install -r requirements.txt
# jupyter nbconvert lab5.ipynb --to html
```

Librerías y Configuración

```
In [5]: import cv2, math, os, random, json
import numpy as np
import matplotlib.pyplot as plt
from tabulate import tabulate
```

```
In [6]: RUN_NAME = "run_experiment_01"
LOG_JSON_PATH = f"{RUN_NAME}.json"
img1_set1 = "images/set1/img1.png"
img2_set1 = "images/set1/img2.png"
logs = {}
```

```
In [7]: def addLog(section, key, value):
    global logs
    if section not in logs:
        logs[section] = {}
    logs[section][key] = value

def saveLogsAsJSON(logsDict, filename):
    with open(filename, "w", encoding="utf-8") as f:
        json.dump(logsDict, f, indent=4, ensure_ascii=False)
    print(f"Logs guardados en: {filename}")
```

```
In [8]: def loadImages(leftPath, rightPath):
    imgLeft = cv2.imread(leftPath)
    imgRight = cv2.imread(rightPath)
```

```

if imgLeft is None or imgRight is None:
    raise ValueError("No se pudieron cargar las imágenes")

return imgLeft, imgRight

```

```

In [9]: def convertToGray(imgLeft, imgRight):
        grayLeft = cv2.cvtColor(imgLeft, cv2.COLOR_BGR2GRAY)
        grayRight = cv2.cvtColor(imgRight, cv2.COLOR_BGR2GRAY)
        return grayLeft, grayRight

```

Código Previo

```

In [10]: def detectWithSift(grayLeft, grayRight, nFeatures=2000):

        # Left se queda fija y right se va a transformar para acoplar con Left
        sift = cv2.SIFT_create(nfeatures=nFeatures)

        kpLeft, desLeft = sift.detectAndCompute(grayLeft, None) # Train (destino)
        kpRight, desRight = sift.detectAndCompute(grayRight, None) # Query (se transforma)

        addLog("features", "kp_left", len(kpLeft))
        addLog("features", "kp_right", len(kpRight))

        return kpLeft, desLeft, kpRight, desRight

```

```

In [11]: def matchWithFlann(desRight, desLeft, ratio=0.7):

        # FLANN KDTree para SIFT
        FLANN_INDEX_KDTREE = 1
        indexParams = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
        searchParams = dict(checks=50)

        flann = cv2.FlannBasedMatcher(indexParams, searchParams)

        # derecha = query, izquierda = train
        matches = flann.knnMatch(desRight, desLeft, k=2)

        # muchos ruido asi que filtramos con Lowe ratio test
        goodMatches = []

        for m, n in matches:
            if m.distance < ratio * n.distance:
                goodMatches.append(m)

        addLog("matching", "good_matches", len(goodMatches))

        return goodMatches

```

```

In [12]: def drawMatchesImage(grayRight, kpRight, grayLeft, kpLeft, goodMatches):

        imgMatches = cv2.drawMatches(
            grayRight, kpRight,

```

```

        grayLeft, kpLeft,
        goodMatches, None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

    plt.figure(figsize=(16,8))
    plt.imshow(imgMatches, cmap="gray")
    plt.axis("off")
    plt.show()

```

```

In [13]: def computeHomography(kpRight, kpLeft, goodMatches):

    # extraer coordenadas x,y
    srcPts = np.float32([kpRight[m.queryIdx].pt for m in goodMatches]).reshape(-1,1)
    dstPts = np.float32([kpLeft[m.trainIdx].pt for m in goodMatches]).reshape(-1,1)

    M, mask = cv2.findHomography(srcPts, dstPts, cv2.RANSAC, 5.0)

    addLog("homography", "inliers", int(mask.sum()) if mask is not None else 0)

    return M

```

```

In [14]: def warpAndBlend(imgLeft, imgRight, M):

    # warp = deformación para calcular nuestro canvas / lienzo

    hLeft, wLeft = imgLeft.shape[:2]
    hRight, wRight = imgRight.shape[:2]

    # ancho = suma de anchos
    # alto = el mayor
    canvasW = wLeft + wRight
    canvasH = max(hLeft, hRight)

    # La derecha se movera hacia coordenadas positivas
    panorama = cv2.warpPerspective(imgRight, M, (canvasW, canvasH))

    # colocar izquierda en el canvas
    panorama[0:hLeft, 0:wLeft] = imgLeft

    # convertir a gris para encontrar zonas negras
    panoramaGray = cv2.cvtColor(panorama, cv2.COLOR_BGR2GRAY)

    _, maskContent = cv2.threshold(panoramaGray, 1, 255, cv2.THRESH_BINARY)
    maskInv = cv2.bitwise_not(maskContent)

    leftOnCanvas = np.zeros_like(panorama)
    leftOnCanvas[0:hLeft, 0:wLeft] = imgLeft

    imgLeftPart = cv2.bitwise_and(leftOnCanvas, leftOnCanvas, mask=maskInv)
    imgRightPart = cv2.bitwise_and(panorama, panorama, mask=maskContent)

    result = cv2.add(imgLeftPart, imgRightPart)

    return result

```

```
In [15]: def cropBlackBorders(img):

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ys, xs = np.where(gray > 0)

    if len(xs) == 0 or len(ys) == 0:
        return img

    xMin, xMax = xs.min(), xs.max()
    yMin, yMax = ys.min(), ys.max()

    return img[yMin:yMax+1, xMin:xMax+1]

In [16]: def stitchPanorama(leftPath, rightPath):

    imgLeft, imgRight = loadImages(leftPath, rightPath)
    grayLeft, grayRight = convertToGray(imgLeft, imgRight)

    kpLeft, desLeft, kpRight, desRight = detectWithSift(grayLeft, grayRight)

    goodMatches = matchWithFlann(desRight, desLeft)

    drawMatchesImage(grayRight, kpRight, grayLeft, kpLeft, goodMatches)

    M = computeHomography(kpRight, kpLeft, goodMatches)

    panorama = warpAndBlend(imgLeft, imgRight, M)

    panorama = cropBlackBorders(panorama)

    panoramaRgb = cv2.cvtColor(panorama, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(16,8))
    plt.imshow(panoramaRgb)
    plt.axis("off")
    plt.show()

    return panorama
```

Task 1

Usted deberá capturar dos sets de imágenes utilizando la cámara de su celular. Para esto deberá crear un set de control y otro experimental. A cada uno deberá ser ejecutado con el código se creo en el workshop. Para cada uno considere:

Set de Control: Rotación Pura (Validación)

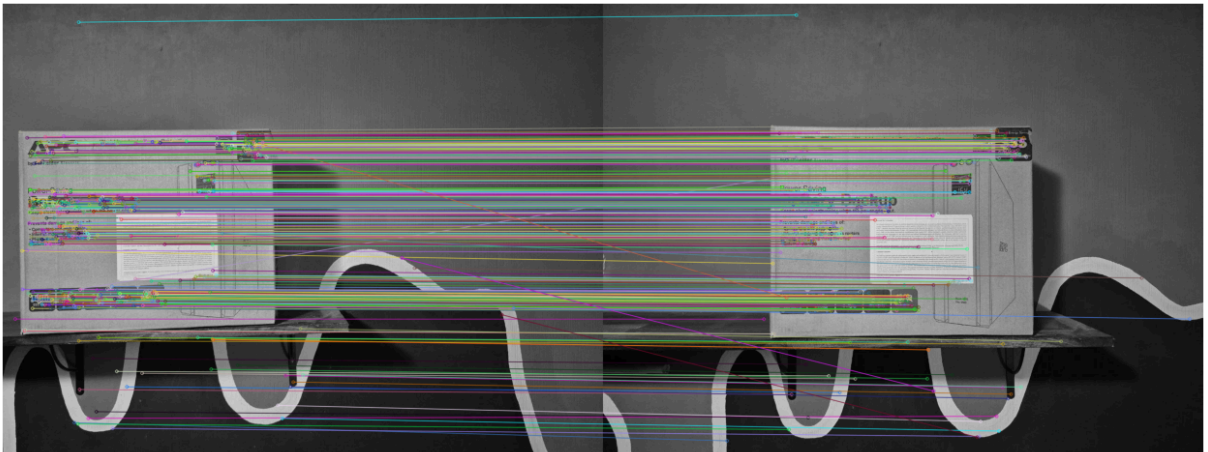
- **Objetivo:** Demostrar que la homografía funciona cuando no hay traslación.
- **Procedimiento:** Desde una posición fija, tome dos fotos de una escena lejana rotando solamente el dispositivo.
- **Resultado Esperado:** Un panorama limpio y continuo.

Set Experimental: Traslación y Profundidad (El Descubrimiento)

- **Escena:** Busque un entorno (pasillo, habitación o exterior) que tenga objetos a distancias claramente variadas.
 - Objeto A (Cercano): A aprox. 50 cm de la cámara (ej. una botella, una silla).
 - Objeto B (Medio): A aprox. 2 metros.
 - Objeto C (Lejano/Fondo): A más de 5 metros (pared, edificio).
- **Procedimiento:**
 1. Tome la **Foto Izquierda**.
 2. Desplácese lateralmente (dé un paso a la derecha) unos 15-30 cm. Mantenga la cámara apuntando al frente.
 3. Tome la **Foto Derecha**.
 - Nota: Asegúrese de que los objetos A, B y C aparezcan en ambas fotos.

```
In [17]: panorama = stitchPanorama(img1_set1, img2_set1)

saveLogsAsJSON(logs, LOG_JSON_PATH)
```





Logs guardados en: run_experiment_01.json

In []:

Task 2

Ejecute su algoritmo de stitching con el **Set Experimental**. Dado que hubo traslación, el algoritmo (usando RANSAC) alineará probablemente el plano dominante (el fondo), pero será incapaz de alinear los objetos cercanos, creando "fantasmas" (duplicados semitransparentes).

Task 2.1 – Visualización del Error

Solicite a su asistente de IA (ChatGPT/Claude/etc) que genere un script para resaltar estas discrepancias, Se le deja este prompt como sugerencia para su uso inicial:

"Actúa como ingeniero de visión artificial. Tengo dos imágenes: la imagen base (izquierda) y la imagen warped (derecha transformada). Escribe un código en Python que genere una imagen compuesta (blending) donde la imagen base se muestre en el canal Rojo y la imagen warped en el canal Cian (Verde+Azul). Esto debe crear un efecto de anaglifo donde los píxeles perfectamente alineados se vean en escala de grises y los errores se vean como bordes de colores."

In []:

notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.

notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.

Task 2.1 – Visualización del Error

Usted deberá medir la distancia en píxeles entre las dos versiones del mismo objeto en su imagen fallida.

- 1. Identifique el Objeto A (Cercano) en la imagen compuesta. Busque un punto distintivo (una esquina, una letra) y encuentre sus coordenadas (x_1, y_1) en la versión roja y (x_2, y_2) en la versión cian.
- 2. Identifique el Objeto B (Medio) y haga lo mismo.
- 3. Identifique el Objeto C (Fondo) y haga lo mismo.
- 4. Puede usar herramientas como matplotlib.pyplot.ginput o simplemente inspeccionar la imagen haciendo zoom en su notebook.

Construya una tabla de resultados como la que se muestra a continuación:

Objeto	Distancia Real Estimada	Coordenada X Fantasma 1	Coordenada X Fantasma 2	Disparidad ($\ x_1 - x_2\ $) en píxeles
A (Cercano)				
B (Medio)				
C (Fondo)				

In []:

notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.
notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.

Task 3

Construya un reporte, donde responda a las siguientes preguntas basándose exclusivamente en sus datos:

- 1. Observe la columna de "Disparidad". ¿Existe una tendencia clara? ¿Cómo se comporta el valor de la disparidad a medida que aumenta la distancia real del objeto?
- 2. Si definimos Z como profundidad y d como disparidad. Basado en su tabla, ¿la relación se parece más a una función lineal ($d = k \cdot Z$) o a una inversa ($d = k/Z$)? Grafique sus 3 puntos si es necesario para visualizarlo.

3. Usted acaba de experimentar con los principios de la Estereoscopia. Explique por qué el algoritmo de Panoramas (Homografía) falló en alinear los objetos cercanos. ¿Qué información tridimensional estaba "escondida" en ese fallo?

In []:

notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.
notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.

In []:

notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.
notebook controller is DISPOSED.

Vea el [registro] de Jupyter (command:jupyter.viewOutput) para obtener más detalles.